

東京 2025 LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

One Config to Rule Them All: From systemd to Reliable Embedded Upgrades

Unifying defaults, overrides, and ephemeral state for embedded and related systems

Behan Webster

東京 2025 Linux Plumbers Conference
TOKYO, JAPAN / DEC. 11-13, 2025



Problem Statement

We've made upgrades atomic, but not predictable.

- Achieve **predictable, reliable, failure-safe full-system upgrades** remains difficult in practice
- Managing `/etc` as **mutable, drift-prone configuration state** introduces persistent upgrade risk
- Supporting **read-only rootfs, factory reset, and compliance requirements** complicates configuration management
- Upgrades still break — even when systems remain bootable and atomic

Core question:

Why does configuration remain the primary source of upgrade risk?

Current Reality

- Use layered paths (`/usr`, `/etc`, `/run`) with drop-ins (systemd)
- Apply **versioned read-only overlays** for `/etc` using confext
- Debian/Fedora use `conf.d` directories for fragments
- Maintain **monolithic configuration files without layering or composition**
- Follow upstream conventions **without consistent configuration strategy**
- Operate in **fragmented ecosystems with many build systems, stacks, and vendor customizations lasting a decade or more**

In short: proven patterns exist, but adoption is uneven and fragmented.

Upgrade Pain in Embedded Systems

- Regulatory and market pressure now **penalizes unreliable upgrade mechanisms**
- Many systems use **A/B or snapshot upgrades** for atomic rollback, but still struggle with configuration drift
- Overwrite `/etc` state during **full-image upgrades**
- Config drift → upgrade failures
- Factory reset logic becomes bespoke
- Rootfs often must be writable
- Risk escalates with fleet scale and uptime requirements

Pattern That Works

- Separate **vendor defaults from local state**
- Compose configuration using **layered fragments**
- Store defaults in **read-only locations** for safe upgrades
- Preserve **local intent independently** from vendor changes
- Generate composed configs **at build or runtime**

Where Current Approaches Fall Short

Related prior art: **NixOS/Guix and A/B snapshot systems** demonstrate declarative, transactional upgrades with rollback and reproducibility, but are not widely adopted in embedded or are not entirely compatible with existing tooling and formats.

- Layering exists, but is **inconsistent and systemd-focused**
- confext enables RO overlays but **not schema, validation, or merge semantics**
- `conf.d` works, but **has no shared structure or composition rules**
- Build systems handle fragments, but **not upgrade-safe configuration** (e.g., unlike the Yocto kernel fragment model, most app configs lack structured merge semantics)
- Apps often use bespoke formats with **no shared model of defaults/overrides**

Core issue:

> Tools exist, but lack **shared semantics for layering, validation, and migration**, especially across app boundaries.

Prior Art: Vyatta-Style Configuration

- Maintain a **central hierarchical configuration model**
- Validate changes before **commit and apply**
- Roll back safely using **transactional edits**
- Generate underlying configs **after validation**

Context: concepts widely used in **network operating systems** to manage complex, interconnected configuration with safety guarantees
Pros: reliability, safety, rollback, schema-driven, app-agnostic
Limitations: assumes writable FS, not layered, no build-time integration
Relevance: complements layered defaults by adding transactional safety
Idea: integrate validation/rollback without breaking legacy systems

Proposed Direction

*Possible building blocks for safer, incremental improvements (compatible with existing approaches like NixOS/Guix and A/B)**

- Produce **/usr defaults from fragments** at build time
- Preserve **site-specific overrides in /etc** as distinct, durable state
- Allow defaults to **evolve without overwriting local intent**
- Provide an optional helper library for **simple path search + merge priority**, not format standardization
- Retain **domain-specific validation/rollback mechanisms domain-specific**
- Apply **layered defaults consistently across images and devices**
- Enable **shared conventions across build systems** without requiring application rewrites shared conventions without requiring application rewrites

Goal: Enable safer upgrades without rewriting formats, replacing tooling, or enforcing uniformity.

Upgrade Pain in Embedded Systems (Examples)

- Ship appliances with **patched, ad-hoc configurations**
- Change config syntax across versions without migration paths
- Lose local overrides or **cause conflicts with new defaults**
- Require safe migration on **read-only rootfs devices**
- Trigger field failures, downtime, costly RMAs

Upgrades should be **boring and reliable by default** — no outages, no DoS, no fear, especially for embedded fleets

We need: layered defaults, preserved overrides, and validation/rollback mechanisms

Trade-offs

- Balance **standardization with flexibility**
- Balance **schema enforcement with developer autonomy**
- Balance **automatic migration with manual control**
- Balance **lightweight tooling with richer frameworks**

Constraints

Reflect realities of embedded systems today, not idealized architectures

- Work with or without systemd
- Support **read-only rootfs and factory reset**
- Tolerate **partial or incremental adoption**
- Preserve **existing configuration formats**

Reality Check

- No universal solution for upgrade safety
- Binary rollback is solved; **config rollback is not**
- Layering, validation, and migration all have cost
- Embedded heterogeneity makes uniform solutions unrealistic

Question: What minimal, compatible conventions can improve outcomes today?

Open Discussion (Part 1): Framing & Drivers

Regulatory, safety-critical, and high-availability environments (e.g., CRA, industrial, medical, automotive) increasingly demand upgrade processes that are predictable, auditable, and failure-safe: but these are not the only drivers. Operational cost, user experience, and product reliability all depend on making upgrades routine, boring, and resilient.

Why this matters

- Regulatory, safety-critical, and high-availability systems increasingly demand **predictable, auditable, failure-safe upgrades**
- Operational cost, user experience, and reliability depend on **boring, routine upgrades**
- These pressures apply broadly: not just to safety-critical deployments

Open Discussion (Part 2): Questions & Context

Key Questions:

- How can build systems play a larger role in achieving safe, predictable upgrades?
- Should build systems converge on conventions for defaults, overrides, and validation?
- What embedded upgrade failure modes matter most to avoid?
- What minimal conventions could improve upgrade safety without major rewrites?
- Should layering be standardized as a convention, a helper library, or left to tooling?
- Where is validation worth centralizing, and where must it remain domain-specific?
- How do we balance incremental improvements with ecosystem fragmentation?

Context:

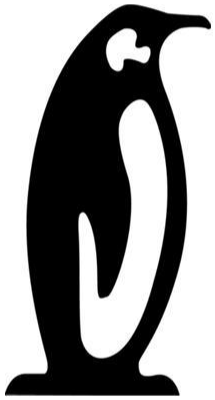
- Build systems already assemble images, defaults, and fragments at scale
- They are a natural integration point for **layered configuration and merge policies**
- A/B rootfs, snapshots, and declarative systems (e.g., NixOS/Guix) demonstrate strong rollback guarantees.
- But across all approaches, **managing configuration state remains the unsolved part of the upgrade problem**.

Goal of the Session

- Unify proven patterns into a **consistent configuration model**
- Apply solutions to **embedded and related Linux ecosystems**
- Identify **practical next steps and collaborators**

Upgrades should **not** be exciting, but boring.

Fear-free upgrades are an ecosystem requirement, not a luxury.



東京 2025
LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

